COMP3331 Assignment Report

*****Note for marker – Read section "where it doesn't work" at the bottom of this report to understand what's wrong with the program.**

Program design overview:

The program implements a forum application in the command line using a combination of TCP and UDP sockets for communication between a client and server.

The forum supports user authentication, thread creation, message posting, message removal/editing, uploading/downloading files to threads, listing threads, reading threads, and logging out. The client sends commands (acceptable commands are displayed to the user) to the server, and the server responds appropriately.

The server can also handle multiple concurrent requests from multiple clients, and it achieves this through multithreading.

Application layer message format:

All communication that is text-based is done over UDP and follows a command-driven format. The user can do the following:

Create thread - CRT <thread_title>

Post message - MSG <thread_title> <message>

Delete msg - DLT <thread_title> <msg_number>

Edit message - EDT <thread_title> <msg_number> <new_msg>

Upload file -UPD <thread_title> <filename>

Download file - DWN <thread_title> <filename>

List threads - LST

Read thread - RDT <thread_title>

Remove thread - RMV <thread_title>

Logout - XIT

The server responds to these commands:

- Saying things like 'Message posted' or 'File uploaded successfully' when the command is successfully carried out
- "SUCCESS" or "READY" for when the client program needs more information before progressing – this is an intermediary step, the command (upload for example) is half done at this point.
- "ERROR <reason>" for invalid operations or unauthorized access.

The server also prints meaningful messages saying things like ‘Request to list threads’, ‘<thread_name> read’ and so on, in response to the commands and their outcomes.

System functionality:

The server:

- Maintains credentials.txt for login/signup.

- Stores active thread files

- Handles UDP requests in a loop via recvfrom().

- Spawns threads to handle concurrent TCP connections during upload/download to prevent blocking the main UDP loop.

- Files are saved alongside thread files and accessed using filenames.

The TCP communication to transfer files occurs after communication between the client and server using UDP. Once both sides are ready to transfer the file, the server replies with "READY" to let the client know that the transfer can begin. The client then establishes a TCP connection to the same server port that’s taken from the command line argument to upload/download the file. The TCP connection closes right after the transfer is complete.

The client:

-Firstly, user is prompter to log in. The signed-up users are stored in credentials.txt and they’re checked for matching passwords for existing users, and new users are prompted to create their password and this then also stored in credentials.txt.

- The client waits and takes user input, sees which one of the commands it matches (if it does match in the first place), and sends to the server over UDP. Invalid commands are rejected.

- Receives error messages and displays user-friendly output.

Multithreading is implemented to handle multiple concurrent requests from multiple clients. This effectively allows the program to continue running despite getting multiple requests from multiple clients. This is because each request is launched in a thread of its own, and it can be processed in the background, without requests being blocked while one’s processing.

I also use locks, as this is needed to ensure that sensitive resources are accessed only by one thread at a time, otherwise it can be used in a conflicting manner. I use locks in server.py whenever I open a file to write to or download/upload.

Design Tradeoffs:

UDP is used for all communications between the server and client except file transfers, and TCP for file transfers

- UDP faster than TCP, however, it's unreliable as it can be lost. However, I implemented a mechanism to recover from losses. If a UDP segment is sent to the server and there's no reply within 2 seconds, it resends the UDP segment.
- TCP is used for file transfers due to its reliability since we don't want file transfers with information missing. However, this is a bit slower than if we used UDP.

Also, when establishing a TCP connection, the client program was very quick to be ready to establish it, while the server took a little longer launching a thread and then establishing the connection. This resulted in the client's TCP connection request falling on deaf ears, so it wasn't establishing a connection. The way I solved this was trying multiple times to establish a connection if it's failing, and making the program sleep momentarily to give the server more time to set up on its side. This solved the problem and the TCP connection is being established reliably.

Where it doesn't work:

The authentication and all 10 commands work as expected, including the upload and download commands. The upload command can upload a file from the client side to the server side, and the download command can download a file from the server side to the client side. These commands all work normally. However, after a UPD or DWN command is made and the file transfer is successful, the next command (lets say its an LST command) will return the same result as before (it will display 'File transfer successful' again). Then the next command (lets say its RDT space) will return the result of the previous command (which was LST), and it will display the active threads. The next command will then read the thread titled 'Space', even if that wasn't the command. In this way, the responses for each command is delayed be one input.

The functionality of the DWN or UPD isn't compromised either. The program downloads and uploads files perfectly. However, the responses become delayed.

This is the only part that doesn't work according to the spec.

Conclusion

In conclusion, the forum achieves the functionality of authentication and the forum operations by using both UDP and TCP effectively, for single and multiple concurrent clients using multithreading.